Nice — I'll walk you through a complete, practical step-by-step guide to deploy your **fitness-website-pipeline-additions-aws** static site using **AWS CodeCommit → CodeBuild → CodePipeline → S3 (optionally CloudFront)**. I'll include CLI commands, example files (buildspec), IAM role / policy examples, and tips for S3 website vs CloudFront. I'll assume your AWS region is us-east-1 in examples; replace with your region if different.

Quick note: do **not** paste real AWS credentials into public chat. The examples below show where and how to configure credentials locally with aws configure. Prefer using IAM roles where possible.

---

0 — Prerequisites

1. AWS account + permissions to create IAM roles, S3 buckets, CodeCommit, CodeBuild, CodePipeline.
2. Locally: aws CLI v2 installed and configured, git installed, Node/npm if your static site needs building.
3. Your project repo directory ready (HTML/CSS/JS or build output like dist/build).
4. Configure AWS CLI locally:

```
aws configure
# Enter:
# AWS Access Key ID [None]: <YOUR_ACCESS_KEY_ID>
# AWS Secret Access Key [None]: <YOUR_SECRET_ACCESS_KEY>
# Default region name [None]: us-east-1
# Default output format [None]: json
```

(Prefer ephemeral credentials/roles in production; for local setup aws configure is fine.) (AWS Documentation)

---

1 — Create two S3 buckets

You'll want:

- **artifact bucket** (for CodePipeline/CodeBuild artifacts)
- **website bucket** (to host the static site)

Bucket names must be globally unique.

If your region ≠ us-east-1:

```
aws s3api create-bucket \
  --bucket my-fitness-artifacts-<unique-suffix> \
  --region us-east-1 \
  --create-bucket-configuration LocationConstraint=us-east-1
```

If **us-east-1**, omit the --create-bucket-configuration:

```
aws s3api create-bucket --bucket my-fitness-artifacts-<suffix> --region us-east-1
aws s3api create-bucket --bucket my-fitness-website-<suffix> --region us-east-1
```

Enable versioning on the artifact bucket (recommended):

```
aws s3api put-bucket-versioning \
  --bucket my-fitness-artifacts-<suffix> \
  --versioning-configuration Status=Enabled
```

If you plan to host directly from S3 (public):

- Enable **Static website hosting** on my-fitness-website-<suffix> and set index document to index.html.
- You can set a bucket policy to allow public read (I'll include a sample policy below). For production, prefer CloudFront + OAI (private bucket) — I cover that later. (AWS Documentation)

---

2 — Create CodeCommit repository and push your code

Create the repo:

```
aws codecommit create-repository \
  --repository-name fitness-website-pipeline-additions-aws \
  --repository-description "Static fitness website"
```

Clone & push your local project:

Option A — using HTTPS Git URL (common):

```
# From your project directory:
git init
git add .
git commit -m "Initial commit"

# Add remote (replace region if needed)
git remote add origin https://git-codecommit.us-east-1.amazonaws.com/v1/repos/fitness-website-pipeline-additions-aws

# Push (you will need HTTPS Git credentials or credential helper)
git push -u origin main   # or master depending on default branch
```

Option B — using git-remote-codecommit (recommended if you use IAM/federated credentials):

```
# Example (no extra git credentials needed if your aws profile has valid creds)
git remote add origin codecommit::us-east-1://fitness-website-pipeline-additions-aws
git push -u origin main
```

For details on CodeCommit setup & credential helpers see AWS docs. (AWS Documentation)

---

3 — Add buildspec.yml to your repo (required by CodeBuild)

Place a buildspec.yml at repository root. Two common approaches:

A) **Pipeline-managed deploy** (recommended): CodeBuild produces an artifact, CodePipeline S3 deploy stage deploys it to website bucket.

Example buildspec.yml (project that builds to build/ — adapt base-directory to dist or . for plain HTML):

```
version: 0.2

env:
  variables:
    ARTIFACT_NAME: "site.zip"

phases:
  install:
    commands:
      - echo "Installing dependencies (if any)"
      - npm ci || true
  build:
    commands:
      - echo "Building site"
      - npm run build || true
  post_build:
    - echo "Packaging build output"
    - cd build || true
    - zip -r ../$ARTIFACT_NAME . || true
    - cd ..

artifacts:
  files:
    - $ARTIFACT_NAME
  discard-paths: yes
```

B) **CodeBuild performs the S3 sync** (simpler if you don't want a separate S3 deploy action):

```
version: 0.2

env:
  variables:
    S3_BUCKET: "my-fitness-website-<suffix>"

phases:
```

```
  install:
    commands:
      - npm ci || true
  build:
    commands:
      - npm run build || true
  post_build:
    - echo "Syncing to S3"
    - aws s3 sync build/ s3://$S3_BUCKET --delete --acl public-read
```

If you use option B, CodeBuild's role must have permission to s3:PutObject etc. Option A uses CodePipeline's S3 deploy action and is cleaner. See AWS samples for hosting CodeBuild output in S3. (AWS Documentation)

---

4 — Create IAM roles & minimal policies

CodeBuild service role trust policy (save to codebuild-trust.json):

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "Service": "codebuild.amazonaws.com" },
    "Action": "sts:AssumeRole"
  }]
}
```

Create role:

```
aws iam create-role --role-name codebuild-fitness-role --assume-role-policy-document
file://codebuild-trust.json
```

Attach an inline policy codebuild-fitness-policy (example gives permissions to read CodeCommit, write to S3, and send logs). Save to codebuild-policy.json and adjust ARNs for your buckets and repo:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject",
        "s3:PutObjectAcl",
        "s3:ListBucket"
      ],
```

```
    "Resource": [
      "arn:aws:s3:::my-fitness-website-<suffix>",
      "arn:aws:s3:::my-fitness-website-<suffix>/*",
      "arn:aws:s3:::my-fitness-artifacts-<suffix>",
      "arn:aws:s3:::my-fitness-artifacts-<suffix>/*"
    ]
  },
  {
    "Effect": "Allow",
    "Action": [
      "logs:CreateLogGroup",
      "logs:CreateLogStream",
      "logs:PutLogEvents"
    ],
    "Resource": "*"
  },
  {
    "Effect": "Allow",
    "Action": [
      "codecommit:GetBranch",
      "codecommit:GetCommit",
      "codecommit:GetRepository"
    ],
    "Resource": "arn:aws:codecommit:us-east-1:<ACCOUNT_ID>:fitness-website-pipeline-additions-
aws"
  }
 ]
}
```

Attach policy:

aws iam put-role-policy --role-name codebuild-fitness-role --policy-name codebuild-fitness-policy --policy-document file://codebuild-policy.json

CodePipeline service role (must be able to call CodeCommit, CodeBuild, S3, and pass the CodeBuild role):

- Create role with trust for codepipeline.amazonaws.com
- Attach a policy allowing codepipeline:* to work on the specific resources and iam:PassRole for the CodeBuild role ARN.

(You can create roles in the Console using the CodePipeline "Create pipeline" wizard which will create the role automatically; that's easiest.) (AWS Documentation)

---

5 — Create CodeBuild project (console or CLI)

Simplest: use the AWS Console → CodeBuild → Create project:

- **Project name**: fitness-site-build
- **Source provider**: AWS CodeCommit → select fitness-website-pipeline-additions-aws
- **Environment**: Managed image (Amazon Linux 2 / standard image), runtime aws/codebuild/standard:..., set appropriate compute (e.g. BUILD_GENERAL1_SMALL).
- **Service role**: use codebuild-fitness-role (or allow CodeBuild to create one and then attach S3/log permissions).
- **Buildspec**: use the buildspec.yml from repo (recommended).
- **Artifacts**: If using CodePipeline, set artifacts type to NO_ARTIFACTS because CodePipeline handles them; if building artifacts for pipeline, choose S3 artifact and name.

You can also create with CLI using a project JSON and aws codebuild create-project — console UI is easier for first run. See CodeBuild docs for examples. (AWS Documentation)

---

6 — Create CodePipeline that ties everything together

Easiest: AWS Console → CodePipeline → Create pipeline:

- Pipeline name: fitness-site-pipeline
- Service role: let console create or supply your codepipeline-role.
- **Source stage**:
  - Provider: **AWS CodeCommit**
  - Repository: fitness-website-pipeline-additions-aws
  - Branch: main (or master)
- **Build stage**:
  - Provider: **AWS CodeBuild**
  - Project: fitness-site-build
- **Deploy stage**:
  - Provider: **Amazon S3**
  - Bucket: my-fitness-website-<suffix>
  - Extract file? set to **Yes** if archive artifact

If you used CodeBuild to run aws s3 sync (option B earlier), you can skip the S3 Deploy action — make pipeline with only Source → Build. If you used CodeBuild to produce artifact zip, configure S3 Deploy stage to extract to website bucket. AWS has a tutorial for creating S3 deploy pipelines. (AWS Documentation)

CLI alternative: build a pipeline.json describing stages and call:

```
aws codepipeline create-pipeline --cli-input-json file://pipeline.json
```

(See AWS docs for the pipeline JSON schema.)

---

7 — Configure S3 website hosting (if using S3 static website)

On the my-fitness-website-<suffix> bucket:

- Properties → Static website hosting → Enable → Index document index.html

Public bucket policy (simple, public-read):

```
{
  "Version":"2012-10-17",
  "Statement":[{
    "Sid":"PublicReadGetObject",
    "Effect":"Allow",
    "Principal":"*",
    "Action":["s3:GetObject"],
    "Resource":["arn:aws:s3:::my-fitness-website-<suffix>/*"]
  }]
}
```

Paste that into bucket policy (only if you accept public access). For production prefer CloudFront + OAI. (AWS Documentation)

---

8 — (Optional but recommended) Use CloudFront + OAI instead of making bucket public

1. Create CloudFront distribution with your S3 bucket as origin.
2. Create an **Origin Access Identity** (OAI) and attach to CloudFront origin.
3. Modify S3 bucket policy to allow only the CloudFront OAI to get objects.
4. Optionally set up a custom domain and ACM certificate for HTTPS.

If you want CodeBuild to invalidate CloudFront cache after deploy, add in post_build:

```
aws cloudfront create-invalidation --distribution-id $CF_DIST_ID --paths "/*"
```

(Grant cloudfront:CreateInvalidation to CodeBuild role.) Using CloudFront gives low latency & HTTPS without making your bucket public.

---

9 — Trigger & test

- Make a change locally, commit and push to CodeCommit:

```
git add .
git commit -m "Update site"
git push origin main
```

- Check CodePipeline console — it should automatically start and show stages Source → Build → Deploy.
- If using S3 hosting, open http://<bucket-website-endpoint>/ or CloudFront domain for production.

You can check pipeline state with CLI:

aws codepipeline get-pipeline-state --name fitness-site-pipeline

---

10 — Troubleshooting & tips

- If pipeline fails in build, check **CloudWatch Logs** for CodeBuild logs (role must allow logs).
- If S3 is blank: make sure artifacts were produced at root (CodeBuild artifacts base-directory matters) or CodeBuild aws s3 sync succeeded. (Repost, Stack Overflow)
- Use CodeBuild environment variable S3_BUCKET rather than hardcoding bucket name.
- Prefer **CodePipeline console wizard** the first time — it auto-creates roles and wiring. (AWS Documentation)

Files you should add to your repo (summary)

- index.html, css/, js/, images/ (your site files)
- buildspec.yml (one of the examples above)
- (Optional) .npmrc, package.json if you build with npm

Minimal example recap (copy-paste)

buildspec.yml (simple "sync to S3" approach)

```
version: 0.2
env:
  variables:
    S3_BUCKET: "my-fitness-website-<suffix>"
phases:
  install:
    commands:
      - echo Installing...
      - npm ci || true
  build:
    commands:
      - echo Building...
      - npm run build || true
  post_build:
    - echo Deploying to s3://$S3_BUCKET
    - aws s3 sync build/ s3://$S3_BUCKET --delete --acl public-read
```

CodeCommit push (local):

```
git init
git add .
```

```
git commit -m "initial"
git remote add origin https://git-codecommit.us-east-1.amazonaws.com/v1/repos/fitness-website-pipeline-additions-aws
git push -u origin main
```

References / further reading

- AWS CodeCommit getting started & setup. (AWS Documentation)
- CodeBuild sample for hosting build output in S3 (static website). (AWS Documentation)
- CodePipeline tutorial for S3 deploy. (AWS Documentation)
- Setup steps for HTTPS / git-remote-codecommit. (AWS Documentation)

If you'd like, I can:

- Generate the exact buildspec.yml, pipeline.json, and IAM policy JSON customized to your **bucket names**, **account ID**, and **build folder name** (build vs dist) and show the aws CLI commands ready to paste — tell me the bucket names and whether you prefer S3-only hosting or CloudFront + OAI and I'll produce those files immediately.